Banking Portal

Core Features, Technical Assumptions & System Limitations

> **Executive Summary:** This document provides a comprehensive technical breakdown of the Angular-based Banking Portal application. It details all implemented functionality organized strictly by project layer (Layer One, Layer Two, Layer Three), documents architectural and domain assumptions derived from the core specification, and outlines system limitations of the client-side demonstration implementation.

Section 1: Feature List

Layer One: Core Foundation & Customer Management

- **Authentication & Guarded Access**:
  - Secure routing powered by `authGuard` and `publicGuard` preventing unauthorized access to banking modules.
  - Dedicated Login view supporting mock credentials for demo accounts (Ahmed Ali `C001`, Mona Hassan `C002`, Elena Rostova `C003`) as well as generic valid email logins.
  - Session persistence via browser `localStorage` (`banking_portal_session`) storing user token, customer metadata, and authentication timestamps.
- **Interactive Banking Dashboard**:
  - Primary landing module presenting real-time account summary indicators, customer selection list, and system-wide portfolio analytics.
  - Dynamic KPI cards displaying Total Portfolio Balance and System Liquidity.
- **Customer Details & Account Portfolio Overview**:
  - Integrated customer directory displaying CIF identification, full name, national ID, customer segment (`Retail`, `Priority`, `Business`), email, and phone contact details.
  - Customer accounts grid showing account type, status (`Active`, `Dormant`, `Frozen`, `Closed`), account ID, masked IBAN, and current balance.
- **Architecture, Services & State Management**:
  - `BankingFacadeService` acting as a single point of truth using Angular Signals (`signal`, `computed`, `asReadonly`) for reactive state management.
  - Data service (`BankingDataService`) implementing HTTP calls with RxJS `shareReplay(1)` caching for static JSON datasets (`customers.json`, `accounts.json`, `transactions.json`, `transaction-types.json`, `transaction-categories.json`).
  - Strong TypeScript type definitions for `Customer`, `Account`, `Transaction`, `TransactionFilter`, `CreateTransactionRequest`, and `UserSession`.
- **Responsive Layout & Visual Design System**:
  - Main application shell (`ShellComponent`) featuring top header navigation bar, active customer indicator, quick account switcher, theme toggle (Dark/Light mode via PrimeNG), and slide-out mobile drawer menu.

- Fully responsive CSS flexbox/grid layout optimized for mobile, tablet, and desktop viewports with glassmorphism visual styling.

Layer Two: Transactions & Ledger Management

- **Transactions Management View**:
  - Comprehensive ledger interface featuring real-time account context switching, active filter badges, transaction counters, and reset controls.
- **Multi-Criteria Filter & Sorting Suite**:
  - **Date Range Filtering**: Filter ledger items by Start Date and End Date.
  - **Transaction Type Filtering**: Filter by `ALL`, `Debit`, or `Credit`.
  - **Category Filtering**: Filter by `ALL` or exact spec categories (`Groceries`, `Bills`, `Shopping`, `Transfer`, `Income`, `Fees`, `Entertainment`).
  - **Keyword Search**: Full-text search matching merchant name or description field.
  - **Sorting**: Order by transaction date or amount in ascending (`asc`) or descending (`desc`) order.
- **Create Transaction Modal & Form**:
  - Modal component (`app-create-transaction`) driven by Angular `ReactiveFormsModule`.
  - Supports field selection for Type (`Debit` / `Credit`), Amount, Date, Merchant Name, Category dropdown, and optional Description textarea.
- **Business Logic & Balance Protection Rules**:
  - **Debit Liquidity Constraint**: Enforces that a `Debit` transaction amount cannot exceed the target account's current balance. Violations immediately trigger a form/facade validation error blocking submission.
  - **Collision-Safe UUID Generation**: Uses native `crypto.randomUUID()` to assign collision-safe unique identifiers to newly recorded transactions, preventing ID collision errors during rapid sequential entries.
  - **Optimistic State Updates**: Successful transactions update account balance and prepend new entries to transaction list in memory instantly without re-fetching remote endpoints.
- **Form Validation & User Experience (UX)**:
  - Custom validators: `noWhitespaceValidator` (prevents whitespace-only text), `maxDecimalsValidator(2)` (enforces max 2 decimal precision), `pastOrTodayDateValidator()` (blocks future dates), and cross-field `debitBalanceValidator()`.
  - Instant inline error messaging, submit loading indicators (`isSubmitting`), and auto-dismiss success alerts.

Layer Three: Advanced Features, Optimization & Data Persistence

- **Mini Statement Component**:
  - Focused recent-activity widget (`app-mini-statement`) embedded on the Accounts page displaying top 5 recent transactions with type icons and color-coded status badges.
- **CSV Data Export & Formula Injection Defense**:
  - Client-side export service (`formatTransactionCsv`, `downloadCsvFile`) enabling users to download transaction ledgers formatted as `.csv` files named with IBAN and timestamp (`transactions-{iban}-{date}.csv`).

    - **Formula Injection Protection**: Explicitly neutralizes security risks by prefixing leading formula trigger characters (`=` , `+` , `-` , `@` , `\t` , `\r`) with a single quote (`'`), preventing spreadsheet formula execution upon opening in Excel/Calc while maintaining full RFC 4180 compliance.
- **Monthly Insights & Financial Analytics**:
    - Analytics breakdown component (`app-monthly-insights`) aggregating monthly total income (credits), total spending (debits), net monthly cashflow, and spending distribution grouped by category.
- **JSON Caching & HTTP Optimization**:
    - Static JSON data endpoints (`assets/mock/`) fetched via `HttpClient` and cached in memory via RxJS `shareReplay(1)` to ensure network payloads are loaded exactly once during session lifecycle.
- **State Persistence & Schema Versioning**:
    - LocalStorage state synchronization (`saveStateToStorage` & `loadStateFromStorage`) using schema versioning (`SCHEMA_VERSION = 'v2'`). If stored schema version mismatch occurs, stale data is safely purged and re-hydrated from baseline JSON files.
- **Virtual Scrolling Ledger & Signal Memoization Performance**:
    - High-performance data table (`p-table`) configured with `[scrollable]="true"` and `[virtualScroll]="true"` (`[virtualScrollItemSize]="54"`) ensuring smooth 60fps rendering even with large transaction datasets.
    - **Computed Signal Memoization**: Utilizes Angular `computed()` signals and `input()` signal inputs across monthly insights, mini statement, transaction filters, and account options to memoize derived state and eliminate redundant re-evaluations during change detection cycles.
- **Loading & Error Handling Ergonomics**:
    - PrimeNG Skeleton screens (`p-skeleton`) rendered during data loading phases.
    - Global inline error alert components (`p-message`) with manual retry handlers (`onRetry()`) to recover gracefully from network or storage failures.

Section 2: Technical Assumptions

1. **Mock Authentication Behavior**: Authentication is modeled strictly for client-side evaluation without a remote Identity Provider or backend JWT service. Any valid email input is accepted and assigned a session context (matching registered mock customers `C001` , `C002` , `C003` or generating a temporary user context), enabling flexible testing without pre-configured password hashes.
2. **Decimal Precision & Rounding Rules**: Currency calculations enforce 2 decimal places using standard floating-point rounding (`Number(val.toFixed(2))`). Transaction input forms restrict entry to a maximum of 2 decimal places (`maxDecimalsValidator(2)`).
3. **Virtual Scrolling Selection**: Per the original specification offering a choice between pagination "or" virtual scrolling, virtual scrolling (`p-table` `[virtualScroll]="true"`) was selected to provide an uninterrupted continuous scroll experience suitable for financial ledgers.
4. **Currency Default**: `EGP` (Egyptian Pound) is established as the default application currency symbol across numeric formatters, table displays, and account summaries, reflecting local banking domain standards.

5. **Schema Field Naming & JSON Structures**: Model interfaces (`Customer` , `Account` , `Transaction`) strictly align with the naming conventions provided in the project JSON specification (e.g., `CIF` capitalized for Customer identifier, `accountId` , `balanceAfter` , `type` as `'Credit' | 'Debit'`).
6. **LocalStorage Schema Versioning**: Schema versioning (`'v2'`) is embedded into `BankingFacadeService` storage keys (`banking_schema_version` , `banking_accounts` , `banking_transactions`). This guarantees automatic migration/clearing if object structures evolve during development.

Section 3: Known Limitations

1. **Mock Authentication Scope**: Authentication runs locally in browser memory without OAuth2/OIDC servers, multi-factor authentication (MFA), or server-validated web tokens.
2. **No Backend Persistence API**: Application data changes (such as newly created transactions or updated balance states) persist exclusively within browser `localStorage` . Clearing browser cache or switching devices resets data back to initial JSON baseline files.
3. **Demo Data Volume**: Pre-populated datasets (`customers.json` , `accounts.json` , `transactions.json`) contain static client-side records designed for demonstration and UI testing purposes.